choosing filters

Table of contents

packages required 1
data prep 1
analyse of the filtered datasets 3

packages required

This analysis depends on the following packages:

data prep

the dataset is loaded and curred. We remove adult = 0 and NAs

```
data250_afabae_00 <- read.csv("../../../../../../01.data/2023_screen_specifics_251genotypes/0
  mutate(ivip = as.factor(ivip)) %>%
  mutate(session = as.factor(session))

data250_afabae_01 <- data250_afabae_00 %>% filter(!is.na(larvae_total_number))
data250_afabae_01$ivip <- droplevels(data250_afabae_01$ivip)

data250_afabae_02<- data250_afabae_01 %>% filter(adult_alive != "0") #remove if no adult
data250_afabae_02$ivip <- droplevels(data250_afabae_02$ivip)

data250_afabae_02$ivip <-as.factor(data250_afabae_02$ivip)
```

Once we have removed the NAs and all rows for which we don't have an adult, we can still make adjustments to the dataset. We can remove outliers to ensure that our distribution approaches a normal distribution, and we can also filter out genotypes based on a threshold

for the minimum number of repetitions required to keep a genotype. We have 3 methods to handle outliers: 1)keep the outliers; 2)remove manually the outliers based on biological reasons; 3) use the interquartile range method (IQR) to remove all data outside a specific range.

The goal of this script is to study how our model reacts to changes in the outlier removal method and the minimum number of repetitions required to retain a genotype.

in the folowing loop we varry two parameters, i the minimal number of reps for the genotypes and the outlier filtering method.

function to calculate all metric used to compare filters

```
analyze_filtered_data <- function(data, i, filter_name) {
  data <- data %>%
    group_by(ivip) %>%
    filter(n() >= i) %>%
    ungroup() %>%
    droplevels()

  if (nrow(data) < 5 || nlevels(data$ivip) < 2) {
    return(NULL)  # skip iterations with too little data
  }

  NGEN <- nlevels(data$ivip)
  n_after <- nrow(data)

  # Fit the model
  mod <- lmer(larvae_total_number ~ ivip + (1|session/block), data = data)
  residuals_values <- residuals(mod)
  RMSE <- sqrt(mean(residuals_values^2))
  shapiro_res <- shapiro.test(residuals_values)$p.value
  shapiro <- shapiro.test(data$larvae_total_number)$p.value

  # Heritability model
  mod_h2 <- lmer(larvae_total_number ~ (1 | ivip) + (1 | session), data = data)
  estimates <- as.data.frame(VarCorr(mod_h2))
  genetic_variance <- estimates$vcov[1]
  residual_variance <- estimates$vcov[3]
  nbE <- nlevels(data$session)
  H2 <- genetic_variance / (genetic_variance + (residual_variance / nbE))

  return(data.frame(
    filter = filter_name, i = i, RMSE = RMSE, n_after = n_after,
    H2 = H2, shapiro = shapiro, shapiro_res = shapiro_res, NGEN = NGEN
```

```
  ))
}
```

analyse of the filtered datasets

```
# Initialize an empty data frame to store results
results <- data.frame(x = numeric(0), i = numeric(0), RMSE = numeric(0),
                      n_after = numeric(0), H2=numeric(0),shapiro=numeric(0),shapiro_res=nu


###filter based on IQR
    # Calculate IQR and bounds for outlier removal
    IQR_value <- IQR(data250_afabae_02$larvae_total_number)
    lower_bound <- quantile(data250_afabae_02$larvae_total_number, 0.25) - 1.5 * IQR_value
    upper_bound <- quantile(data250_afabae_02$larvae_total_number, 0.75) + 1.5 * IQR_value
    filter ="IQR"
    # Filter out outliers
    data_iqr <- data250_afabae_02[data250_afabae_02$larvae_total_number >= lower_bound & data


for (i in 2:9) {
  res <- analyze_filtered_data(data_iqr, i, filter_name = "IQR")
  if (!is.null(res)) results <- rbind(results, res)
}

for (i in 2:9) {
  res <- analyze_filtered_data(data250_afabae_02, i, filter_name = "no_filter")
  if (!is.null(res)) results <- rbind(results, res)
    }


  # Identify the indices of the top 3 highest values
top_3_indices <- order(data250_afabae_02$larvae_total_number, decreasing = TRUE)[1:3]
    # View the top 3 highest values
data250_afabae_02[top_3_indices, ] %>% dplyr::select(genotype, ivip, aphid, larvae_total_numb
```

```
     genotype       ivip         aphid larvae_total_number
1146      164 IVIP101838 A.fabae AF001                 100
1089      155 IVIP101738 A.fabae AF001                  84
123        19 IVIP100234 A.fabae AF001                  77
```

```r
# Remove the rows with the top 3 highest values
data250_afabae_03 <- data250_afabae_02[-top_3_indices, ]

data_manual<- data250_afabae_03 %>% filter(larvae_total_number != "0")

for (i in 2:9) {
  res <- analyze_filtered_data(data_manual, i, filter_name = "Manual_filter")
  if (!is.null(res)) results <- rbind(results, res)
}

print(results)
```

```
          filter i      RMSE n_after        H2      shapiro  shapiro_res NGEN
1            IQR 2  9.233038    1735 0.3447490 5.266510e-11 1.483460e-08  215
2            IQR 3  9.229700    1729 0.3446498 5.543759e-11 1.270126e-08  212
3            IQR 4  9.226764    1714 0.3428978 7.173332e-11 1.658017e-08  207
4            IQR 5  9.261827    1686 0.3366220 1.084763e-10 1.885992e-08  200
5            IQR 6  9.344007    1621 0.3188691 2.352926e-10 1.694455e-08  187
6            IQR 7  9.363300    1519 0.2900334 6.309009e-10 1.945748e-08  170
7            IQR 8  9.474470    1309 0.2334308 8.692906e-09 1.509873e-08  140
8            IQR 9  9.729894     917 0.2094154 8.087895e-07 2.549500e-06   91
9      no_filter 2 10.256889    1766 0.2957885 3.470226e-14 1.326403e-13  215
10     no_filter 3 10.258422    1762 0.2964816 3.127027e-14 1.274759e-13  213
11     no_filter 4 10.216287    1747 0.3001616 2.337939e-14 1.340294e-13  208
12     no_filter 5 10.220998    1719 0.2985800 5.058276e-14 2.058611e-13  201
13     no_filter 6 10.284203    1654 0.2870996 1.875066e-13 7.813558e-13  188
14     no_filter 7 10.339821    1570 0.2682935 5.961660e-13 1.494831e-12  174
15     no_filter 8 10.373956    1367 0.2267790 3.134460e-12 3.314494e-12  145
16     no_filter 9 10.644708     999 0.2130584 6.541015e-10 3.209480e-10   99
17 Manual_filter 2  9.634853    1750 0.3377797 4.862441e-11 3.124783e-10  215
18 Manual_filter 3  9.635051    1746 0.3384563 4.206604e-11 2.953583e-10  213
19 Manual_filter 4  9.584852    1728 0.3444121 2.348789e-11 1.660786e-10  207
20 Manual_filter 5  9.625195    1700 0.3385120 3.469540e-11 1.947332e-10  200
21 Manual_filter 6  9.724575    1640 0.3199037 8.005854e-11 2.647481e-10  188
22 Manual_filter 7  9.760221    1544 0.3039618 4.653498e-10 5.248763e-10  172
23 Manual_filter 8  9.839597    1334 0.2571395 4.052453e-09 7.448826e-10  142
24 Manual_filter 9 10.119513     966 0.2086034 5.997644e-07 9.699518e-08   96
```

Based on these results it seems appropriate to select the IQR filter as it yield best heritability, lower RMSE and more normal data.

```
write.csv(x = results,
          file = "filter_annalysis.csv",
          quote = F, row.names = F)
```

```
iqr_results <- results %>% filter(filter == "IQR")
```

We now need to visualize the data we generated from the IQR filter to choose the best value for i (minimal number of repetitions to keep a genotype)

Based on these first results it appears that IQR is the best filtering option. We still need to detremine the best value for i. For that we will use a similar loop to which we will add the nucleotid diversity.

```
# Initialize an empty data frame to store results
results <- data.frame(x = numeric(0), i = numeric(0), RMSE = numeric(0),
                      n_after = numeric(0), H2=numeric(0),shapiro=numeric(0),shapiro_res=nu


for (i in 2:9) {
  ###filter based on IQR
    # Calculate IQR and bounds for outlier removal
    IQR_value <- IQR(data250_afabae_02$larvae_total_number)
    lower_bound <- quantile(data250_afabae_02$larvae_total_number, 0.25) - 1.5 * IQR_value
    upper_bound <- quantile(data250_afabae_02$larvae_total_number, 0.75) + 1.5 * IQR_value
    filter ="IQR"
    # Filter out outliers
    data <- data250_afabae_02[data250_afabae_02$larvae_total_number >= lower_bound & data250_


    data <- data %>%
      group_by(ivip) %>%           # Group by genotype
      filter(n() >= i) %>%             # Keep only groups with i or more repetitions
      ungroup()


    data$ivip <- droplevels(data$ivip)

    NGEN<-nlevels(data$ivip)
```

```r
# Number of rows after removing outliers
n_after <- nrow(data)



# Fit the model
mod <- lmer(larvae_total_number ~ ivip + (1|session/block), data = data)

# Extract fitted values and residuals
fitted_values <- fitted(mod)
residuals_values <- residuals(mod)

# Perform the Shapiro-Wilk test
shapiro_result <- shapiro.test(residuals_values)

# Print the p-value (this is your quantification of normality)
shapiro_res <- shapiro_result$p.value

# Perform the Shapiro-Wilk test
shapiro_result <- shapiro.test(data$larvae_total_number)

# Print the p-value (this is your quantification of normality)
shapiro <- shapiro_result$p.value


# Calculate RMSE (Root Mean Squared Error)
RMSE <- sqrt(mean(residuals_values^2))

# Fit the model with a fixed formula
mod_h2 <- lmer(larvae_total_number ~ (1 | ivip) + (1 | session), data = data)

# Get the variance components
estimates <- as.data.frame(VarCorr(mod_h2))

# Extract the relevant variance components
genetic_variance <- estimates$vcov[1]
residual_variance <- estimates$vcov[3]  # Adjust based on the placement of residual varia

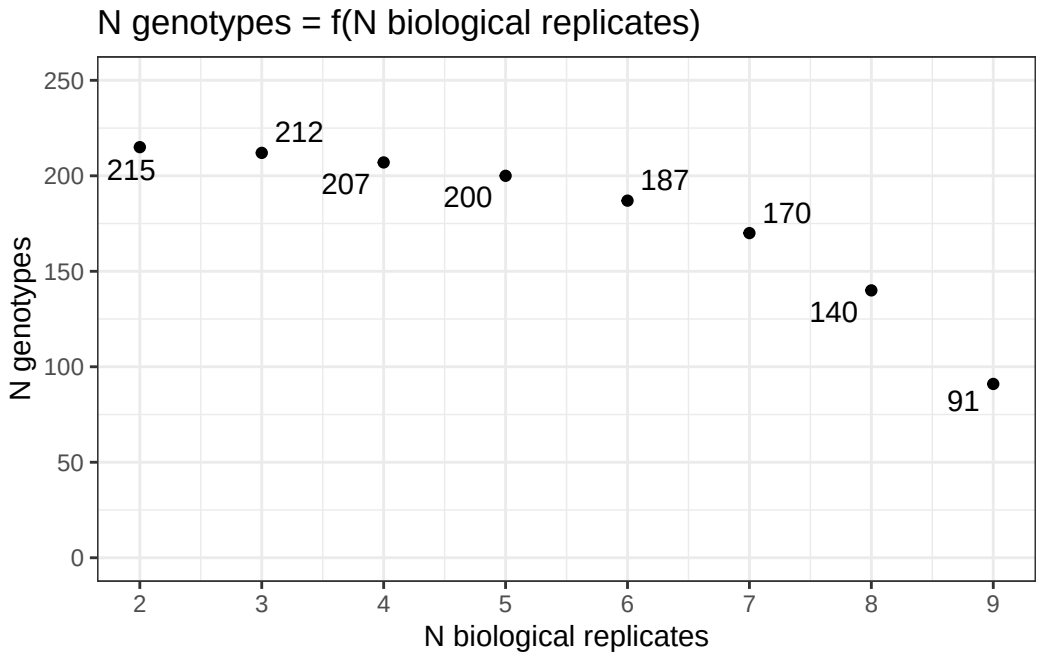
nbE <-nlevels(data$session)

# Calculate heritability (h2)
H2 <- genetic_variance / (genetic_variance + (residual_variance / nbE))
```

```
    # Store the results in the data frame
    results <- rbind(results, data.frame(filter = filter, i = i, RMSE = RMSE, n_after = n_af
}
```

We now need to visualize the data we generated to choose the best filter

```
ggplot(results, aes(x = i, y = NGEN)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(0,250)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = NGEN), arrow = T) +
  labs(x = "N biological replicates", y = "N genotypes") +
  ggtitle("N genotypes = f(N biological replicates)")
```

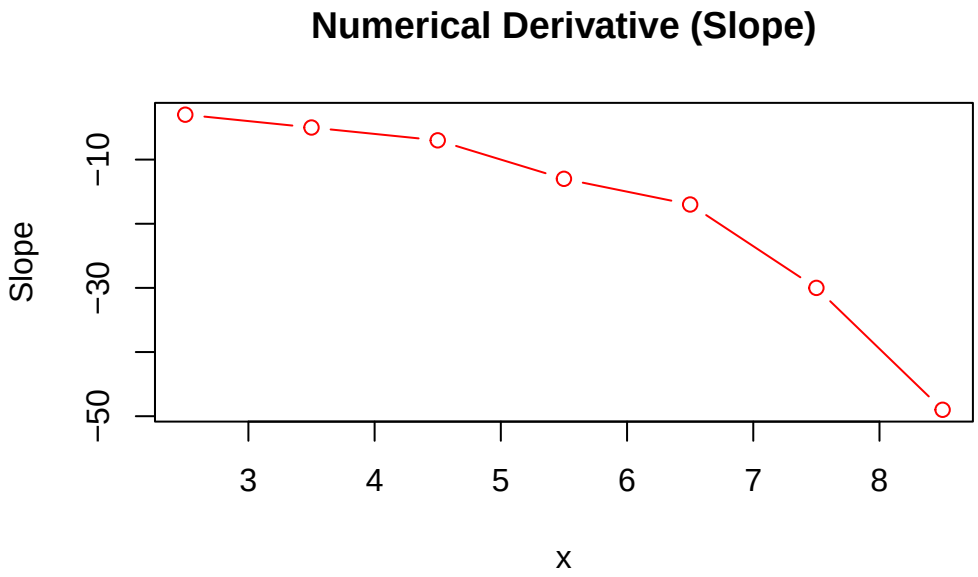



```
y<-as.vector(results$NGEN)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)
```

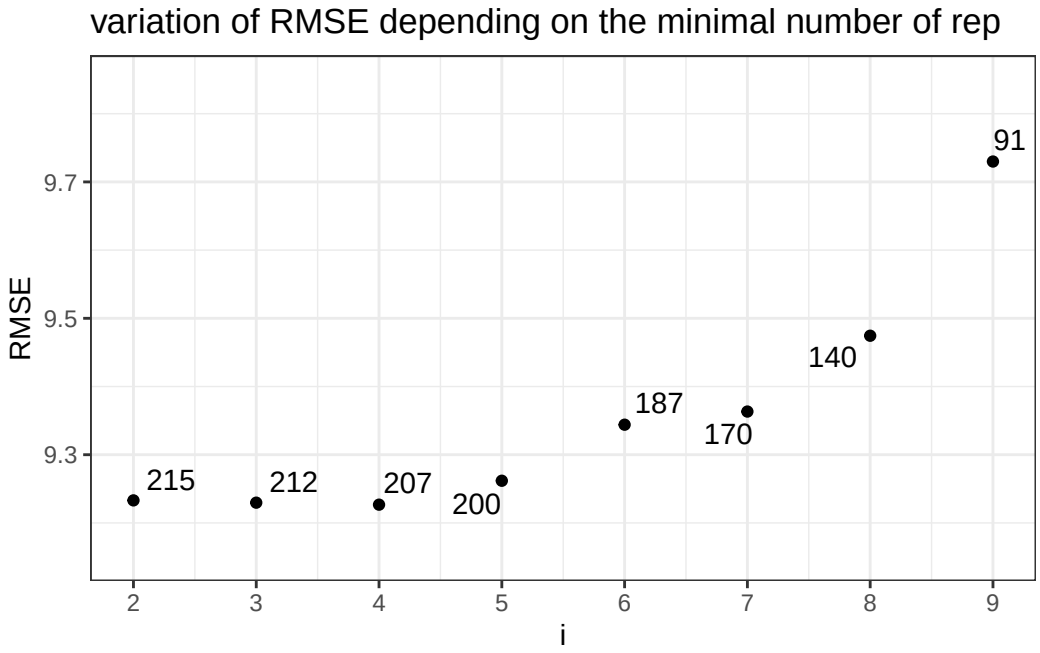
```r
# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



It can be hard to read the relation between Genotype number and i. Therefore it is helpfull to plot the slop (the value is the difference between number of genotype with i and i + 1) based on the decrease in genotypes number we can choose 5 or 7 as the best i value.

```r
ggplot(results, aes(x = i, y = RMSE)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(9.15,9.85)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = NGEN), arrow = T) +
  labs(x = "i", y = "RMSE") +
  ggtitle("variation of RMSE depending on the minimal number of rep")
```

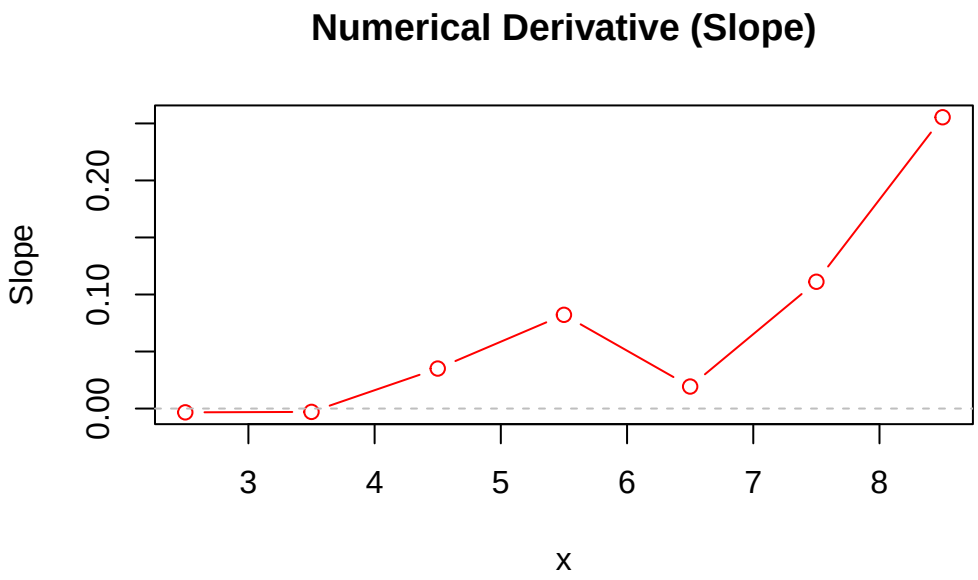



```r
y<-as.vector(results$RMSE)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

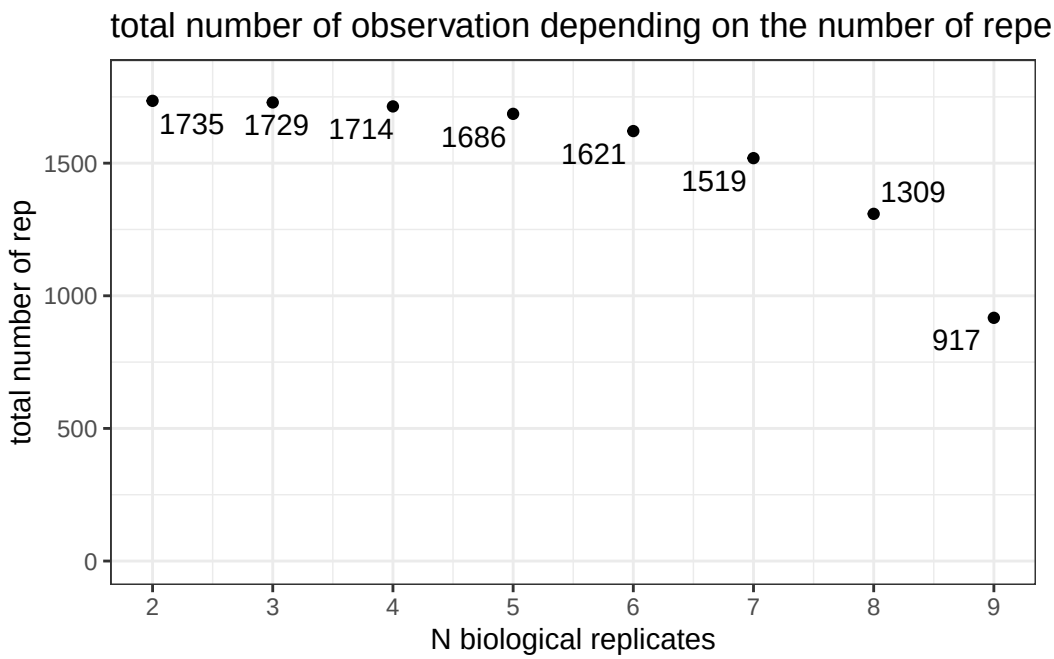
# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



the lowest value for RMSE is with i=4. However base on the curve we could be tempted to choose a value of 5 or 7 for i.

```
ggplot(results, aes(x = i, y = n_after)) +
  geom_point() +
  scale_x_continuous(breaks=seq(1,13,1)) +
  scale_y_continuous(limits = c(0,1800)) +
  theme_bw() +
  ggrepel::geom_text_repel(aes(label = n_after), arrow = T) +
  labs(x = "N biological replicates", y = "total number of rep") +
  ggtitle("total number of observation depending on the number of repetitions")
```

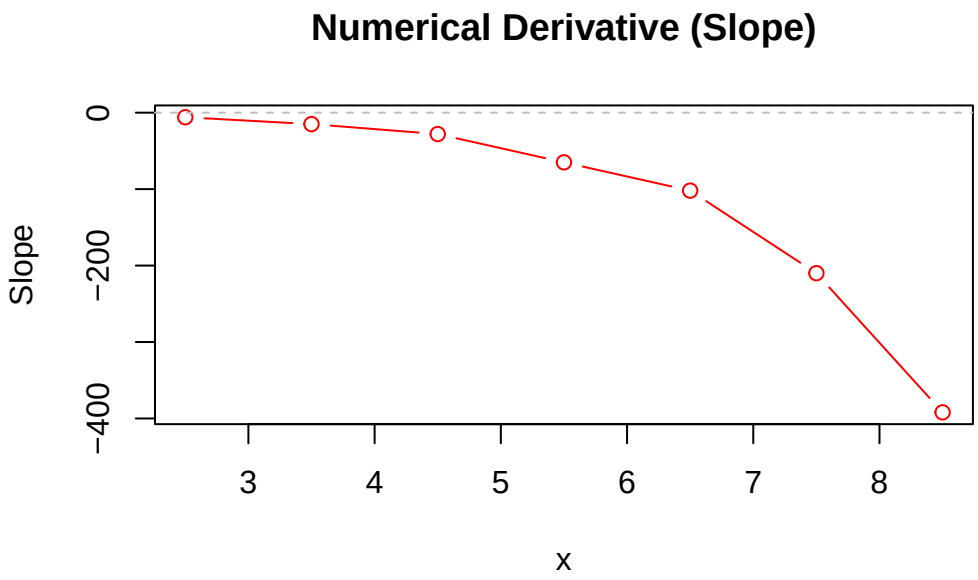



```r
y<-as.vector(results$n_after)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

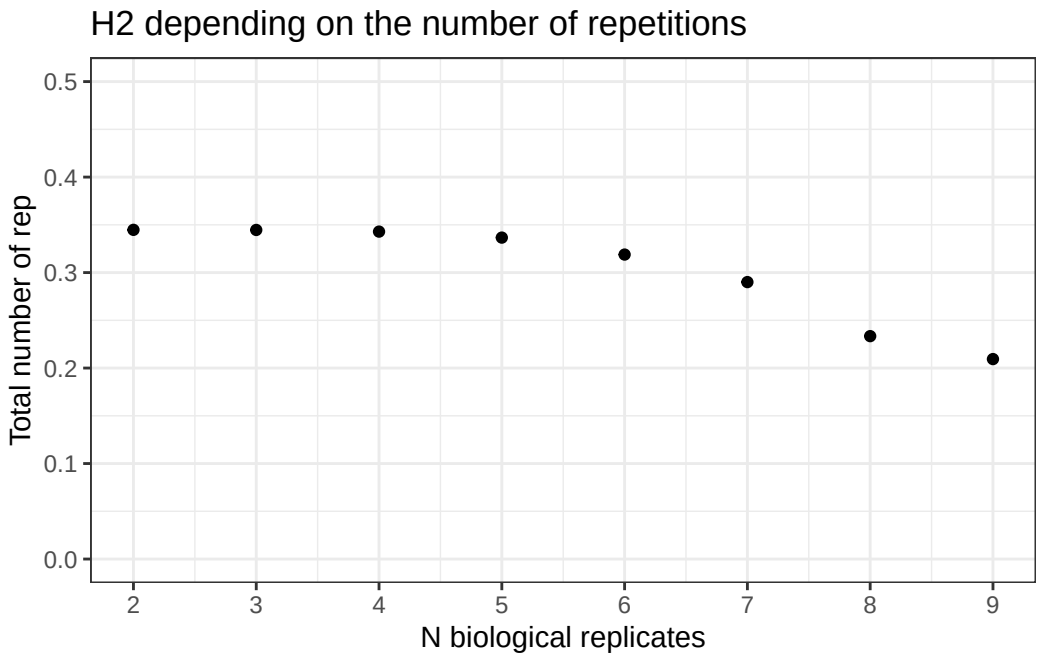
# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



based on overall number of notation we could choose 5 or 7.

```
ggplot(results, aes(x = i, y = H2)) +
  geom_point() +
  scale_x_continuous(breaks = seq(1, 13, 1)) +
  scale_y_continuous(limits = c(0, 0.5)) +
  theme_bw() +
  labs(
    x = "N biological replicates",
    y = "Total number of rep"
  ) +
  ggtitle("H2 depending on the number of repetitions")
```

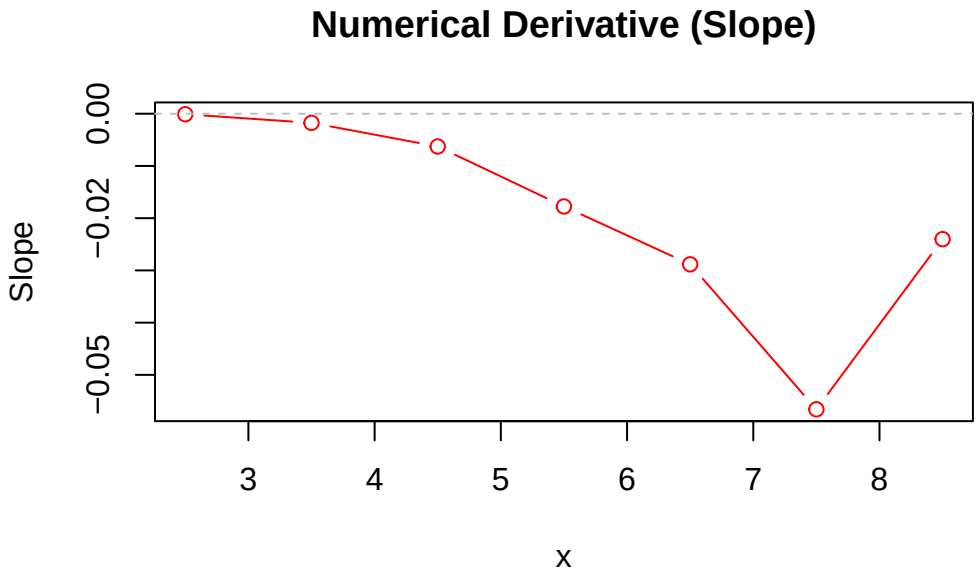



```r
y<-as.vector(results$H2)
x<-as.vector(results$i)

# Compute slope (finite differences)
slope <- diff(y) / diff(x)

# To align slopes with x-values, use midpoints
x_mid <- (x[-1] + x[-length(x)]) / 2

# Plot slope (derivative)
plot(x_mid, slope, type = "b", col = "red", main = "Numerical Derivative (Slope)",
     ylab = "Slope", xlab = "x")
abline(h = 0, col = "gray", lty = 2)  # reference line for zero slope
```



based on H2 the best option are 5 or 7